PayCore: Payment Service & Ledger — Complete Design

A Java 21 payment platform: a transactional core with a double-entry ledger and idempotent APIs, a Kafka-compatible event bus, signed webhooks, a CQRS read model and reconciliation, fronted by a Go gateway. It runs end to end on a laptop with one command and proves its correctness through scripted failure drills and committed test evidence.

1. What it is and why it works for your resume

PayCore is a miniature Razorpay/Stripe backend built as a small platform. Merchants create payments through an API behind a gateway, payments move through a strict lifecycle against a simulated bank/PSP, and every rupee movement is recorded in a double-entry ledger that can be mathematically verified. State changes are published as versioned events to a Kafka-compatible bus, and independent services deliver signed webhooks, maintain a read model for dashboards, and reconcile the books against a settlement file. The whole system is instrumented and chaos-tested, runs entirely on a local machine (Docker Compose or k3d), and ships with committed, reproducible evidence for every claim.

Why this project and not another one:

- **It is the one domain where correctness beats features.** Interviewers at Razorpay/Flipkart-type companies ask about double charging, retries, crashes between DB commit and event publish, and concurrent updates. Here every answer is something you built and tested.
- **It complements what you already have.** Blob-Cloud demonstrates storage integrity and the Go gateway demonstrates traffic handling and resilience. PayCore demonstrates transactional consistency, and with the gateway in front of it, two projects become one coherent system.
- **It covers the backend vocabulary honestly.** Relational modelling and locking, caching, event streaming, CQRS, observability, local Kubernetes deployment and failure testing, each tied to a measured result rather than a keyword.
- **It is measurable.** Invariants (ledger drift = 0), concurrency tests, and crash and failover drills give you hard numbers for resume bullets.

Explicit non-goals (say this in the README, it shows judgement): no cloud hosting (everything runs locally), no real card data (PCI scope), no real bank integration, no multi-currency FX, no service mesh, no custom Kubernetes operators, no microservice per

entity. Kafka (via Redpanda), Redis and Kubernetes (k3d) are in scope, but only because each has a stated reason and a measured result (section 6).

2. Functional requirements

FR1. Merchant onboarding & auth. Create merchant; issue API key (stored only as a hash); register webhook URL and secret.

FR2. Payment lifecycle. Create payment intent (amount in minor units, currency INR, metadata); authorize; capture (full); void an authorization; refund (full or partial, multiple partials allowed up to the captured amount).

FR3. Idempotency. Every mutating endpoint accepts an `Idempotency-Key`. Same key + same body returns the original response; same key + different body returns 422; concurrent same-key requests execute once.

FR4. Double-entry ledger. Every capture, refund, fee and payout posts a balanced transaction (sum of debits = sum of credits) to an append-only ledger. Merchant balance is queryable and provably derived from entries.

FR5. Settlement/payout. Periodic job creates payouts that move merchant payable balance to a simulated bank account.

FR6. Webhooks. Events (`payment.authorized` , `payment.captured` , `payment.failed` , `refund.processed` , `payout.created`) delivered to merchant URL, HMAC-signed, retried with backoff, visible delivery log, manual replay endpoint.

FR7. Reconciliation. Ingest a PSP settlement file (CSV), match against ledger by PSP reference, report missing/extra/amount-mismatched rows.

FR8. Simulated PSP. A separate mock acquirer service with configurable latency, failure rate, timeouts and duplicate/late responses. This is your fault-injection tool and what makes the chaos tests possible.

FR9. Query & audit APIs. Get payment, list payments (cursor pagination), get ledger entries for a payment, get merchant balance, get webhook deliveries.

FR10. Event streaming. Every state change publishes a versioned Protobuf event through the outbox to the bus, keyed by payment ID with a per-payment sequence number. Consumers are independent, idempotent and replayable; failures flow through retry topics to a DLQ with a replay tool.

FR11. Merchant read model (CQRS). Dashboards, list and summary APIs read from a projection built from the event stream. It can be rebuilt from scratch from the topic and verified against the ledger.

FR12. Edge protection. Traffic enters through the Go gateway, which enforces per-merchant rate limits, circuit breaking, load balancing and health checks, and forwards the request ID and trace context.

FR13. Operability. Health and readiness endpoints, metrics, distributed traces, an invariant-check endpoint, backups with a tested restore, and a runbook for each failure drill.

3. Non-functional requirements

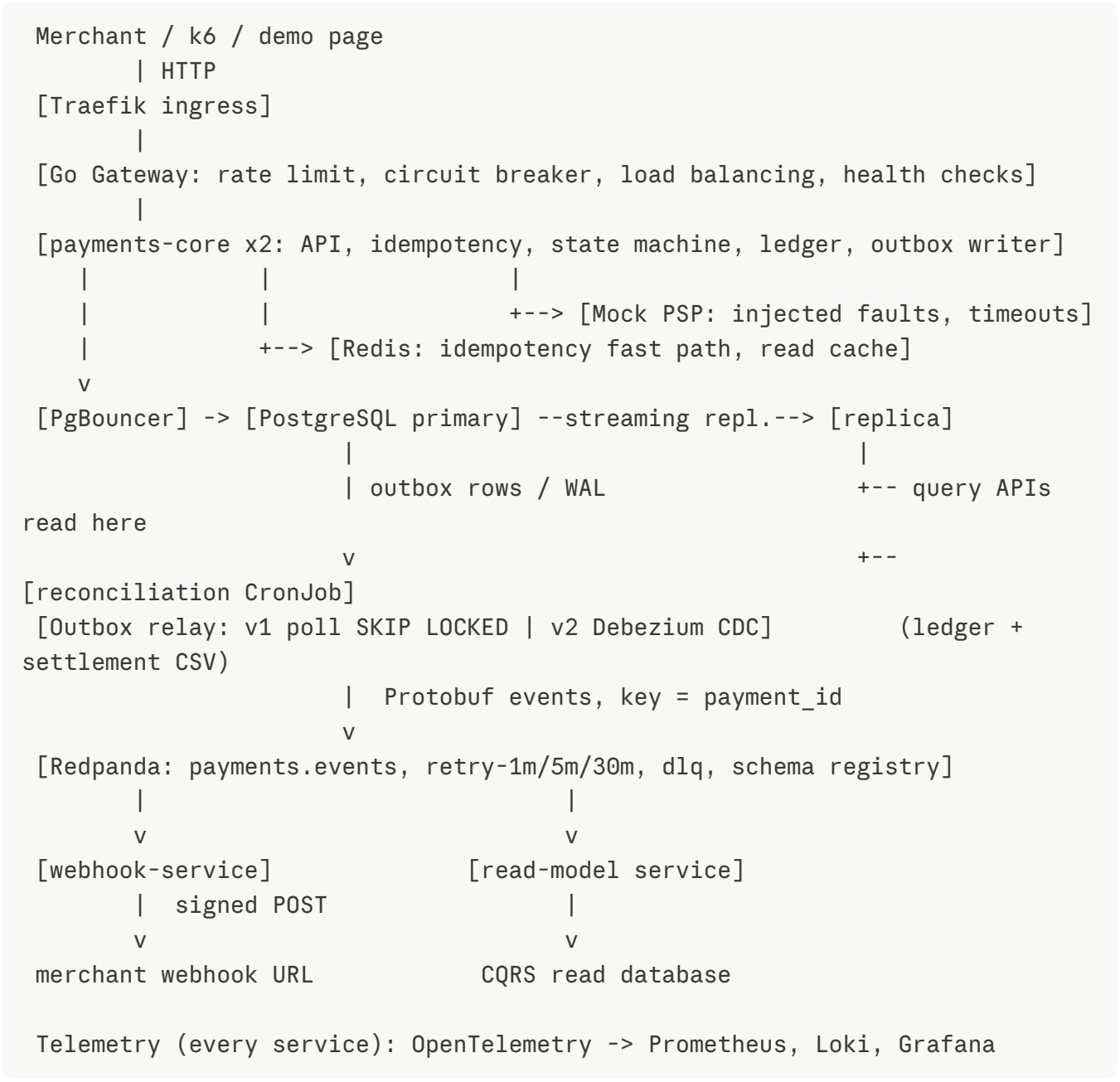
Area	Requirement	How it's verified
Correctness	Ledger always balanced; no payment in an illegal state; no double capture/refund beyond amount	Invariant checker + property tests + TLA+ model
Exactly-once effect	Retried or concurrent requests, and redelivered events, never duplicate money movement or side effects	Concurrent idempotency test, consumer dedupe tests
Durability	A committed payment/event is never lost across app, relay, consumer or DB-primary crash	`kill -9` chaos tests, failover drill
Consistency	Payment state change, ledger entries and outbox event commit atomically	Single DB transaction + crash test
Latency	p99 under 150 ms for create/capture through the gateway at target load on the full profile (record the hardware, then claim your real number)	k6
Throughput	Report measured laptop numbers with their hardware, and lead with relative A/B results that hold on any machine	k6 A/B variants
Event pipeline	p99 commit-to-topic latency under 1 s; consumer lag bounded under load; no event lost or applied twice	Lag metrics, consumer crash tests, relay comparison

Area	Requirement	How it's verified
Resilience	PSP timeouts/failures handled without stuck money; unknown outcomes resolved automatically	Fault-injection tests against mock PSP
Availability	Survives loss of one payments-core replica, one consumer, Redis, or the DB primary; DB failover target under 60 s (record the real figure)	Failure drills (section 8.7)
Recoverability	Restore from backup into a fresh cluster with zero ledger drift; document RPO/RTO and the sync vs async replication trade-off	Restore drill
Evolvability	Backward-compatible schema and event changes; zero-downtime updates	Protobuf compatibility checks, expand/contract migration under load
Observability	Metrics, structured logs with correlation IDs, one distributed trace from gateway to consumer	Grafana dashboards, trace assertion test
Security	API key hashing, HMAC webhooks with timestamp, constant-time compares, no secrets in logs, SSRF protection, rate limiting per merchant at the gateway	Unit/integration tests
Maintainability	Module boundaries enforced; services interact only through events and published APIs	ArchUnit and Gradle dependency rules
Efficiency	Lite profile fits in about 4-6 GB, full profile in about 12-16 GB	Resource dashboard, `make doctor`
Reproducibility	Any claim can be reproduced from a clean clone with one command; every result is committed with its hardware, versions and seeds	`make verify`, `make evidence`, `EVIDENCE.md`, CI reruns

Be honest in the README about the scale ceiling: one Postgres primary with a replica, vertically scaled, and a single-node Redpanda that production would replace with three brokers. "Here is how I would shard by merchant" is a stronger answer than pretending it scales infinitely.

4. High-level design

4.1 Architecture



4.2 Key architectural decisions (and the reasoning you give in interviews)

1. **Modular core, extracted edges.** The money path (payments, ledger, idempotency, outbox writer) is one deployable on one database; splitting it would turn every capture into a distributed transaction. Separable consumers (webhook delivery, read model, reconciliation) and the mock PSP run as their own services, and you can say exactly why the boundary sits there.

2. **PostgreSQL as the single source of truth.** One ACID store means payment state, ledger entries and outbox events commit atomically. This is the whole point of the outbox pattern.
3. **Plain SQL over JPA for the money path.** Use Spring `JdbcClient` or jOOQ with explicit SQL, `SELECT ... FOR UPDATE`, and `INSERT ... ON CONFLICT`. JPA hides the locking and flush behaviour you want to demonstrate understanding of.
4. **Never hold a DB transaction across a network call.** The PSP call happens between two short transactions. This is the most common real-world mistake and a great talking point.
5. **Outbox first, then a bus.** The outbox row commits with the state change, so there is no dual write. A relay publishes to Redpanda (Kafka API). Version 1 polls with `FOR UPDATE SKIP LOCKED`; version 2 tails the WAL with Debezium; you benchmark the two against each other. Consumers are idempotent through a `processed_events` table.
6. **Append-only ledger.** Entries are immutable (enforced with a DB trigger that rejects UPDATE/DELETE). Corrections are new reversing transactions.
7. **Java 21 virtual threads for blocking I/O** (PSP calls, webhook HTTP). Know the caveat: your real concurrency limit becomes the Hikari connection pool, so size it deliberately and explain why.
8. **Postgres stays authoritative over Redis.** Redis only accelerates idempotency lookups and reads. If it dies or returns stale data, correctness does not change; only latency does.

4.3 Core flows

Create payment. The client calls `POST /v1/payments` through the gateway with an `Idempotency-Key`. The idempotency filter checks Redis first, then claims the key in Postgres. The service inserts a `CREATED` payment plus an outbox event in one transaction, and the stored response is returned on any retry.

Authorize + capture (the money path).

1. Tx 1: compare-and-set `CREATED -> AUTHORIZING` (`UPDATE ... WHERE id=? AND status='CREATED'`; zero rows updated means someone else is already handling it). Commit.
2. Call the PSP with a deterministic PSP idempotency reference (`psp_ref = payment_id + attempt_no`), a timeout, and retry only on safe failures.
3. Tx 2 on success: `AUTHORIZING -> AUTHORIZED`, insert attempt row, outbox `payment.authorized`. On decline: `-> FAILED`, outbox `payment.failed`.
4. Capture: Tx 3 locks the payment row, checks `AUTHORIZED`, posts the ledger transaction, moves to `CAPTURED`, inserts outbox `payment.captured`, all in one commit.

Unknown outcome (PSP timeout). The payment stays `AUTHORIZING`. The resolver job queries the PSP by `psp_ref` for payments stuck beyond a threshold and finalizes them. You never guess, never double-charge, and never leave money in limbo. This single scenario is worth an entire interview answer.

Refund. Locks the payment row, validates `refunded_total + amount <= captured_amount`, posts reversing ledger entries, updates the running total, emits `refund.processed`. Concurrent refunds are serialized by the row lock.

Event delivery. The outbox row commits with the state change. The relay publishes it to the `payments.events` topic (Protobuf, keyed by `payment_id` so each payment's events stay ordered). The webhook service consumes the topic, signs and POSTs to the merchant, and on failure republishes to retry topics (1m, 5m, 30m, 2h, 6h) and finally a DLQ. Every consumer records handled event IDs in `processed_events`, so redelivery from Kafka is harmless.

5. Low-level design

5.1 Data model (PostgreSQL)

All money is `BIGINT` in minor units (paise). Never floats or decimals-as-doubles. All IDs are UUIDv7 or ULID (time-ordered, index friendly) generated in the app.

```
-- payments-core database (each service owns its own database)
merchants(id, name, api_key_hash, webhook_url, webhook_secret, created_at)

accounts(id, merchant_id NULL, type, currency, name)
  -- type: ASSET | LIABILITY | REVENUE | EXTERNAL
  -- system accounts: PSP_RECEIVABLE, PLATFORM_FEE_REVENUE (optionally
sharded), BANK
  -- per-merchant:    MERCHANT_PAYABLE

account_balances(account_id PK, balance BIGINT, version BIGINT, updated_at)
  -- projection, updated in the same tx as entries; verified against
SUM(entries)

payments(id, merchant_id, amount BIGINT, currency, status, captured_amount,
         refunded_amount, psp_ref, version, event_seq BIGINT, created_at,
updated_at)
  -- CHECK (refunded_amount <= captured_amount)
  -- event_seq: per-payment counter, incremented with every outbox event

payment_attempts(id, payment_id, attempt_no, psp_ref, outcome, latency_ms,
created_at)
```

```
ledger_transactions(id, payment_id NULL, kind, created_at, posted_by)
  -- kind: CAPTURE | REFUND | PAYOUT | REVERSAL

ledger_entries(id, txn_id, account_id, direction, amount BIGINT, created_at)
  -- PARTITION BY RANGE (created_at), monthly partitions
  -- direction: DEBIT | CREDIT, amount > 0, append-only (trigger)
  -- deferred constraint trigger: per txn, SUM(debit) = SUM(credit)

idempotency_keys(merchant_id, key, request_hash, status, response_code,
                 response_body, locked_until, created_at,
                 PRIMARY KEY (merchant_id, key))

outbox_events(id, aggregate_id, event_seq, event_type, schema_version,
              payload, trace_context, created_at, published_at NULL)
  -- PARTITION BY RANGE (created_at); partial index WHERE published_at IS
NULL
  -- v1 relay claims rows with SKIP LOCKED; v2 (Debezium) reads the WAL,
  -- and a retention job drops old partitions

payouts(id, merchant_id, amount, status, created_at)

-- webhook-service database
webhook_deliveries(id, event_id, merchant_id, attempt, status,
next_attempt_at,
                   last_status_code, last_error, created_at)
processed_events(consumer, event_id, processed_at, PRIMARY KEY (consumer,
event_id))

-- read-model-service database
payment_view(payment_id PK, merchant_id, status, amount, captured_amount,
             refunded_amount, last_event_seq, updated_at)
  -- apply an event only if event_seq = last_event_seq + 1;
  -- ignore if <= last_event_seq; on a gap, wait or re-fetch
merchant_daily_summary(merchant_id, day, captured, refunded, fees,
payment_count)
processed_events(consumer, event_id, processed_at)

-- reconciliation database
recon_runs(id, file_name, started_at, matched, missing, mismatched)
recon_discrepancies(id, run_id, psp_ref, kind, expected, actual)

-- Redis keys (cache only, never authoritative)
idem:{merchant_id}:{key}         completed response + request_hash, TTL 24h
cache:payment:{id}:v{version}    payment JSON, invalidated by version bump
cache:balance:{merchant_id}      short TTL, refreshed from the read replica
```

Ledger postings (the accounting model, kept deliberately small):

Event	Debit	Credit
Capture of 1000, fee 20	PSP_RECEIVABLE 1000	MERCHANT_PAYABLE 980, PLATFORM_FEE_REVENUE 20
Refund of 400 (fee not returned, a stated policy)	MERCHANT_PAYABLE 400	PSP_RECEIVABLE 400
Payout of 500	MERCHANT_PAYABLE 500	BANK 500

Invariants: global sum of debits equals sum of credits; `MERCHANT_PAYABLE` never goes negative; `account_balances` equals the sum of entries for every account.

Event contract (Protobuf). Events are the second public API of the system, so they are versioned and checked in CI.

```
message PaymentEvent {
  string event_id       = 1;   // UUIDv7, the consumer dedupe key
  string event_type     = 2;   // payment.captured, refund.processed, ...
  uint32 schema_version = 3;
  string payment_id     = 4;   // Kafka message key
  string merchant_id    = 5;
  uint64 event_seq      = 6;   // per payment, gapless
  google.protobuf.Timestamp occurred_at = 7;
  map<string, string> trace_context = 8;   // W3C traceparent, carried across
the bus
  oneof payload {
    PaymentAuthorized authorized = 20;
    PaymentCaptured   captured   = 21;
    RefundProcessed   refunded   = 22;
    PayoutCreated     payout     = 23;
  }
}
```

Rules: only add optional fields, never reuse a field number, and fail CI on any incompatible change.

5.2 Idempotency algorithm (the part interviewers push on)

```
on request with (merchant_id, key):
  hash = SHA-256(method + path + canonical body)

  -- 1. Redis fast path (an optimisation only; Postgres is the authority)
  r = GET idem:{merchant_id}:{key}
  if r is COMPLETED and r.request_hash == hash   -> replay stored response
  if r is COMPLETED and r.request_hash != hash   -> 422
  on any miss or Redis error                     -> fall through to Postgres

  -- 2. Postgres claim (authoritative)
  INSERT INTO idempotency_keys (..., status='IN_PROGRESS', request_hash=hash,
                                locked_until = now()+30s)
         ON CONFLICT DO NOTHING
  if inserted:      execute handler, then UPDATE status='COMPLETED' + store
response
                    (in the SAME transaction as the business change when
possible);
                    after commit, SET the Redis entry with a 24h TTL
  else:
    load existing row
    if request_hash differs           -> 422 key reuse with different body
    if COMPLETED                      -> replay stored response (same status
+ body)
    if IN_PROGRESS and not expired    -> 409 or short wait-and-poll
    if IN_PROGRESS and lock expired   -> safe takeover (handler is itself
idempotent
                                         via state-machine compare-and-set)
```

Discuss why the business change and the idempotency record should commit atomically, and what happens if you cannot (PSP call in the middle): the state machine's compare-and-set plus the PSP reference make the retry safe.

5.3 Concurrency control

- **Payments:** optimistic `version` column plus compare-and-set status updates for state transitions; `SELECT ... FOR UPDATE` on the payment row for capture/refund where you need to read-then-write.
- **Ledger accounts:** when a transaction touches multiple accounts, lock `account_balances` rows **in ascending account_id order** to make deadlocks impossible. Add a retry on SQLSTATE `40001` / `40P01` as a safety net.

- **Isolation:** run at `READ COMMITTED` with explicit locks. Document why you did not use `SERIALIZABLE` everywhere (retry storms under contention) and where it would be the right call.
- **Hot account problem:** the platform fee account is touched by every capture. Implement both versions (a single fee account and N sharded fee sub-accounts summed on read) and report throughput for each under the same k6 load.
- **Consumers:** each consumer applies an event and records `processed_events` in one transaction. The read model also checks `event_seq`, so duplicates and reordering caused by retries cannot corrupt state.
- **Failover:** when the Postgres primary changes, in-flight transactions fail and connections reset. Clients retry with the same idempotency key and the state machine's compare-and-set makes the retry safe. Test exactly this.
- **Redis:** never part of correctness. Every Redis failure path falls through to Postgres.

5.4 Webhooks

- Payload signed as `HMAC-SHA256(secret, timestamp + "." + body)`; headers `X-PayCore-Signature`, `X-PayCore-Timestamp`, `X-PayCore-Event-Id`, and the payload carries `event_seq`.
- Delivery guarantee is **at-least-once**; consumers dedupe by event id. State this clearly; claiming exactly-once delivery is a red flag.
- Retries use Kafka retry topics (1m, 5m, 30m, 2h, 6h) read by a delay-aware consumer that pauses the partition until a message is due, with jitter, then a DLQ after the last attempt and a replay tool that republishes from it. A `webhook_deliveries` row per attempt powers the merchant-visible delivery log.
- Retries can reorder deliveries for one payment. Document it and let merchants order by `event_seq`.
- SSRF protection: block private/loopback IP ranges when merchants register URLs and again at delivery time.

5.5 Reconciliation

The reconciliation service runs as a Kubernetes CronJob in the full profile (or on upload in either profile) and reads the ledger from the replica. Input CSV: `psp_ref, amount, status, settled_at`. For each row, look up the payment by `psp_ref`. Discrepancy kinds: `MISSING_IN_LEDGER`, `MISSING_IN_PSP_FILE`, `AMOUNT_MISMATCH`, `STATUS_MISMATCH`. The mock PSP can deliberately generate a file with injected discrepancies so you can show the job catching them.

5.6 API surface

```
POST /v1/merchants                    (admin)
POST /v1/payments                     create intent
POST /v1/payments/{id}/authorize
POST /v1/payments/{id}/capture
POST /v1/payments/{id}/void
POST /v1/payments/{id}/refunds
GET  /v1/payments/{id}
GET  /v1/payments?cursor=&limit=      served by the read model
GET  /v1/payments/{id}/ledger
GET  /v1/merchants/me/balance
GET  /v1/merchants/me/summary         read model: totals by status and day
GET  /v1/webhook-deliveries
POST /v1/webhook-deliveries/{id}/replay
POST /v1/payouts                      (or scheduled)
POST /v1/admin/reconciliation         upload settlement CSV
GET  /v1/admin/invariants             runs the ledger invariant checker
GET  /v1/admin/dlq                    list dead-lettered events
POST /v1/admin/dlq/{id}/replay
GET  /v1/admin/consumers/lag
GET  /healthz  /readyz  /metrics      every service
```

Publish an OpenAPI spec (springdoc) and keep errors in a consistent problem+json shape. The Protobuf event schemas are the second public contract and are published alongside it.

5.7 Package layout

```
paycore/                          (Gradle multi-module)
  Makefile                        doctor, up-lite, up-full, chaos, load,
verify, evidence
  platform-common/                ids, money, clock, errors, tracing, generated
Protobuf events
  payments-core/                  deployable
    merchant/                     auth filter, api keys
    idempotency/                  Redis fast path, Postgres store, hashing
    payment/                      domain (state machine), service, repo,
controller
    ledger/                       posting engine, balances, invariant checker
    psp/                          client, circuit breaker
    outbox/                       writer, polling relay (v1)
  outbox-cdc/                     Debezium connector config and comparison
harness (v2)
  webhook-service/                deployable: consumer (bus, or direct-outbox
mode in lite),
```

```
                                signer, retry topics, DLQ, replay
  read-model-service/           deployable: projector, query API, rebuild
command
  reconciliation/               deployable job: settlement ingest, matcher
  mock-psp/                     deployable: configurable fault injector
  e2e-tests/                    Testcontainers, chaos, jqwik, k6 scripts
  deploy/                       Compose files, Helm charts or Kustomize, k3d
scripts,
                                CloudNativePG, Redpanda, optional Argo CD
apps
  evidence/                     EVIDENCE.md and committed run folders
(section 8.6)
  docs/                         per-component decision notes, runbooks,
restore drill
  formal/                       TLA+ spec and TLC configuration
```

The `ledger` package exposes one method to the rest of `payments-core`: `post(LedgerTransaction)`. Enforce with ArchUnit that nothing else touches ledger tables. Services never import each other; they interact only through Protobuf events and published APIs, enforced with Gradle dependency rules.

6. Tech stack

The money path stays one transactional unit on PostgreSQL. Everything around it earns its place with a stated reason and a measured result, listed in the last column.

Layer	Choice	Why it is here	Proof you publish
Runtime	Java 21 (virtual threads), Spring Boot 3	Product-company standard; modern concurrency to discuss	Hikari pool-size sweep: throughput and p99 vs pool size, with and without virtual threads
Data access	jOOQ or `JdbcClient`, Flyway	Explicit SQL and locking; versioned, backward-compatible migrations	Expand/contract migration run during a live load test
System of record	PostgreSQL 16 on CloudNativePG (primary + replica), PgBouncer	ACID, `SKIP LOCKED`, constraint triggers, automatic failover	Kill the primary under load, then run the invariant checker: zero drift

Layer	Choice	Why it is here	Proof you publish
Table growth	Monthly partitions on `ledger_entries` and `outbox_events`; replica for query APIs	Keeps hot indexes small and isolates read load	`EXPLAIN ANALYZE` before and after on 10M+ entries
Event bus	Redpanda (Kafka API) with schema registry	Fan-out to independent consumers, per-payment ordering, replay	Consumer-lag graph; rebalance and broker-restart tests
Outbox relay	v1 polling with `SKIP LOCKED`; v2 Debezium CDC	The standard outbox pattern, built two ways	Publish latency, DB load and failure behaviour compared head to head
Event contracts	Protobuf, registry compatibility checks in CI	Safe schema evolution	CI fails on a breaking schema change
Cache	Redis	Idempotency fast path and read cache only	Hit rate and p99 delta; kill Redis mid-test, correctness unchanged
Edge	Your Go gateway	Rate limiting, circuit breaking, load balancing, health checks	End-to-end load test through the gateway
Resilience	Resilience4j on the PSP client	Timeouts, retries on safe failures, circuit breaker, bulkhead	Mock-PSP fault matrix
Observability	OpenTelemetry, Prometheus, Grafana, Loki	One trace across gateway, API, Kafka and consumer (context in Kafka headers)	Published trace and dashboard
Testing	JUnit 5, Testcontainers, jqwik, Toxiproxy, k6, ArchUnit, TLA+/TLC	See 6.4 and section 7	Reproducible commands in the README
Delivery	Docker Compose, k3d, Makefile, GitHub Actions CI, optional Argo CD	See section 8	Scripted failure drills with committed evidence

6.1 Service boundaries

Service	Owns	Scaling
payments-core	API, idempotency, state machine, ledger, outbox writer, PSP client	Horizontal replicas, one database
webhook-service	Delivery, retry topics, DLQ, replay	Consumer group, scaled by lag
read-model-service	CQRS projection for merchant dashboards and list APIs	Consumer group, own schema
reconciliation	Settlement-file matching, discrepancy report	Kubernetes CronJob
mock-psp	Configurable latency, failures, duplicate responses	Single replica

Payments and the ledger are not split on purpose: they must commit in one transaction.

6.2 Event flow

1. A state change, its ledger entries and its outbox row commit in one transaction.
2. The relay publishes a Protobuf event to `payments.events`, keyed by `payment_id`. Keying by `merchant_id` would order a whole merchant but create hot partitions; per-payment order is what correctness needs, so events carry a per-payment sequence number for consumers that care.
3. Each consumer checks `processed_events(consumer, event_id)` and applies its work in the same transaction, so redelivery is harmless.
4. Failures go to retry topics with backoff, then to a DLQ with a replay tool. A poison message never blocks a partition.
5. Alert on consumer lag and DLQ depth.

6.3 Redis, used narrowly

- **Idempotency fast path:** `SET key NX PX` as first-line dedupe. The Postgres `idempotency_keys` row stays the authority, and a Redis miss or outage falls through to it.
- **Read cache:** balances and payment lookups with versioned keys, invalidated by events from the bus.

- **Not used for:** distributed locks guarding money, or as a source of truth. Locking is Postgres row locks.
- **Measure honestly:** if the hit rate or latency gain is small, report that. "Not worth it on write paths, worth it on reads" is a stronger result than an unmeasured claim.

6.4 Formal model (optional, high signal)

Write a small TLA+ spec of the payment state machine, the idempotency protocol and the outbox, and model-check it with TLC for: no double capture, refunds never exceed capture, balances conserved, and every committed event eventually published. Then make your Testcontainers and jqwik tests assert the same invariants against the real implementation.

6.5 Still skipped

Service mesh, Elasticsearch, MongoDB, custom Kubernetes operators, a microservice per entity, Redis locks around money, and any claim of exactly-once delivery. If an interviewer asks about one of them, the answer you want is "I considered it, and here is why not."

7. Testing strategy (this is what produces your resume numbers)

Every resume claim should map to a test you can re-run in front of an interviewer.

1. **Concurrent idempotency.** Fire 500 parallel requests with the same key. Assert exactly one payment row, one ledger transaction, and 500 identical responses. Repeat with different bodies to assert 422s, and repeat with Redis disabled and with Redis killed mid-test to prove it only affects latency.
2. **Double-capture / double-refund races.** 200 concurrent captures on one payment: exactly one succeeds. 100 concurrent partial refunds that together exceed the captured amount: total refunded never exceeds it. Include a deliberately naive implementation (check-then-update without a lock) on a branch to show the test fails there. That contrast is your strongest talking point.
3. **Ledger property-based tests (jqwik).** Generate random sequences of create, capture, refund, payout and fault events across many merchants. After each run assert: every transaction balances, global debits equal credits, no negative merchant payable, and balances equal the sum of entries.
4. **Crash consistency.** Testcontainers run the app and Postgres. Kill the app with `kill -9` at randomized points (after Tx1, during the PSP call, mid-Tx3, between commit and relay), restart, let the resolver and relay finish, then run the invariant checker. Run it a thousand times in a loop and report "N injected crashes, 0 invariant violations, 0 lost events."

5. **Fault injection on the PSP.** Mock PSP configured for timeouts, 500s, slow responses and duplicate callbacks. Assert no stuck `AUTHORIZING` payments after the resolver runs and no double charges.
6. **Network faults.** Toxiproxy between app and DB, and between services and the bus (latency, resets, partitions), to verify retry and transaction behaviour.
7. **Outbox relay, both versions.** Kill the polling relay mid-batch and restart the Debezium connector mid-stream; assert no event is lost. Then benchmark the two head to head: commit-to-topic latency percentiles, DB CPU and load, and behaviour when the broker is down (including WAL growth from a stalled replication slot).
8. **Consumer correctness.** Kill consumers mid-batch, force rebalances, and redeliver duplicates; assert `processed_events` dedupes and effects apply once. Send a poison message and watch it reach the DLQ and replay cleanly. Run a merchant endpoint that fails 70% of the time and assert every event is eventually delivered at least once with valid signatures.
9. **Read-model correctness.** Rebuild the projection from offset 0 and assert it equals the live projection and the ledger totals. Deliver events duplicated and out of order and assert `event_seq` handling keeps it consistent.
10. **Database failover under load.** Kill the CloudNativePG primary pod during a k6 run. Record failover time and the error window, assert no committed payment is lost, clients recover through idempotent retries, and the invariant checker reports zero drift.
11. **Schema and deploy safety.** Protobuf compatibility checks in CI; run an expand/contract migration and a rolling update during live load with zero failed requests (and, if Argo Rollouts is enabled, a deliberately bad release that rolls back automatically).
12. **Load test (k6) through the gateway.** Mixed workload (for example 60% create+authorize+capture, 25% reads, 15% refunds) at ramping rate. Record p50/p95/p99, TPS, error rate, DB pool saturation and consumer lag, then run the invariant checker and require drift of exactly zero. Run variants: Redis on/off, single vs sharded fee account, pool-size sweep, virtual threads on/off.
13. **Tracing.** An integration test asserts that one trace ID spans gateway, API, outbox, bus and webhook consumer.
14. **Formal model.** TLC passes on the TLA+ spec (no double capture, refunds bounded, balances conserved, committed events eventually published), and the tests above assert the same invariants on the implementation.
15. **Backup and restore.** Restore into a fresh cluster, run the invariant checker, and record the restore time.

16. **Architecture tests.** ArchUnit and Gradle rules enforcing that only the ledger package touches ledger tables and that services never import each other, with CI failing on violations.

Resume bullets this can support (only write numbers you actually measured):

- Built a payment platform in Java 21, PostgreSQL, Redpanda (Kafka API) and Redis, with a double-entry ledger, idempotent APIs and a transactional outbox; sustained **X TPS at p99 Y ms** through a Go gateway on a laptop-class k3d cluster with **zero ledger drift**.
- Verified exactly-once money effects under **N concurrent same-key requests** and **N injected crashes** (app, relay, consumer, DB primary) using Testcontainers, Toxiproxy and property-based testing, with no duplicate charges or lost events.
- Implemented the outbox two ways (`SKIP LOCKED` polling and Debezium CDC) and benchmarked latency and DB load; added Protobuf event contracts with CI compatibility checks, idempotent consumers, retry topics and a DLQ.
- Packaged the system to run end to end locally (Docker Compose and a 3-node k3d cluster with CloudNativePG failover: **Z s** failover, zero committed-data loss in drills) with scripted chaos drills and CI-published, reproducible test evidence.
- Model-checked the payment state machine and idempotency protocol in TLA+ and asserted the same invariants in integration tests.

Pick the three or four strongest for the resume and keep the rest in the README. Write only numbers you actually measured.

8. Local-first runtime, demo and evidence

Everything runs on one machine with no cloud account. One command brings the system up, one command runs each failure drill, and every claim in the repo is backed by committed, reproducible evidence. Nothing depends on a free-tier provider, so nothing can be reclaimed, rate-limited or go offline during a demo.

8.1 Run profiles

Profile	Command	Contents	Memory	Use
lite	`make up-lite`	Docker Compose: payments-core, Postgres, Redis, mock PSP, webhook service reading the outbox directly (no Kafka), Prometheus and Grafana	~4-6 GB	Daily development, a quick interviewer clone, most tests

Profile	Command	Contents	Memory	Use
full	`make up-full`	k3d cluster (1 server, 2 agents): CloudNativePG primary and replica, PgBouncer, Redpanda, Debezium, Redis, all services, Go gateway, OpenTelemetry, Prometheus, Grafana, Loki	~12-16 GB	Failover and Kafka drills, trace demo
ci	GitHub Actions	Test suites on Testcontainers; nightly heavier drills	Runner limits	Independent evidence

Check your RAM before committing to `full`. On an 8 GB machine, drop Loki and Debezium locally and run the heavy drills in CI. `make doctor` checks RAM, Docker, ports and tool versions before anything starts.

8.2 Command contract

```
make doctor      check RAM, Docker, ports, versions
make up-lite     Compose stack
make up-full     k3d cluster with the whole system
make down        tear everything down
make seed        demo merchants, API keys, sample payments
make demo        scripted happy path with narration
make chaos       scripted failure drills, each ending in an invariant check
(PASS/FAIL)
make load        k6 workloads and A/B variants
make verify      unit, integration, property, crash and model-check suites
make evidence    collect results into evidence/runs/<date>-<sha>/
```

8.3 The k3d cluster

```
k3d cluster: 1 server + 2 agents
  agent-1   CloudNativePG primary, payments-core, Go gateway, Redpanda, Redis
  agent-2   CloudNativePG replica, payments-core, webhook, read-model,
            reconciliation, Debezium
  server    Traefik ingress, Prometheus, Grafana, Loki
```

- Helm charts or Kustomize overlays with resource requests and limits, readiness and liveness probes, and PodDisruptionBudgets.

- Images are pinned and pre-imported with `k3d image import`, so nothing is pulled from the internet at demo time.
- Anti-affinity keeps the Postgres primary and replica on different agents.
- Redpanda runs as a single node. Say plainly that it is a single point of failure here and that production would run three brokers with replication factor 3.
- Secrets are generated dev values from `.env.example`; no real secrets exist in this system.
- Argo CD and Argo Rollouts are optional. Add them only after everything else works, for the GitOps and automatic-rollback story.

8.4 Data tier and backups

- **PostgreSQL:** CloudNativePG with one primary and one replica, PgBouncer in front, and a logical replication slot for the Debezium relay.
- **Backups:** base backups plus WAL archiving to a local MinIO (S3-compatible) container. The restore drill restores into a fresh cluster, runs the invariant checker on it, and records the restore time.

8.5 CI as independent evidence

GitHub Actions is free for public repositories and Testcontainers runs on its runners.

1. **Every push:** unit tests, Testcontainers integration tests, jqwik property tests with logged seeds, ArchUnit, Protobuf compatibility checks, the TLC model check, and a short k6 smoke run.
2. **Nightly:** the heavier chaos and load suites in a throwaway k3d cluster on the runner, with results uploaded as artifacts.
3. **Labelling:** CI numbers are labelled as CI-hardware numbers and never mixed with your laptop numbers.

8.6 Evidence pipeline

```
evidence/
  EVIDENCE.md                 claim -> test -> command -> committed result
  runs/<date>-<sha>/
    environment.json          CPU, RAM, OS, tool versions, git SHA
    seeds.txt                 jqwik, chaos and k6 seeds
    k6/                       raw JSON + generated summary
    chaos/                    one log per drill + invariant-checker output
    tlc/                      model-check output
    grafana/                  dashboard JSON + screenshots
    traces/                   sample trace export
  videos/                     demo and drill recordings
```

- `make evidence` generates the run folder; results are never hand-edited.
- Every number in the README or on the resume links to a run folder.
- Lead with relative A/B results (Redis on vs off, polling vs Debezium, single vs sharded fee account, pool-size sweep). They hold on any hardware; absolute laptop TPS does not.
- Commit one canonical run per release tag and keep raw data compressed.
- `EVIDENCE.md` is the file an interviewer actually reads. Each resume claim gets one row: the test, the command that reproduces it, and the committed result.

8.7 Failure drills to script and record

1. Kill the Postgres primary pod under load; show automatic failover, then zero ledger drift.
2. Kill Redpanda and a consumer mid-batch; show no lost or duplicated effects.
3. Inject a poison message; show it land in the DLQ and replay cleanly.
4. Drain a node; show the PodDisruptionBudgets and rescheduling working.
5. Restore from backup into a fresh cluster and run the invariant checker.
6. Optional with Argo Rollouts: ship a deliberately bad release and show the automatic rollback.

8.8 Live demo script (about 8 minutes)

1. **Pre-flight:** `make doctor`, the stack already up, Grafana open.
2. **Happy path:** create, authorize and capture a payment with `curl`; show the ledger entries, the signed webhook arriving, and the read-model dashboard updating.
3. **Idempotency:** replay the same key and show the identical response; fire a concurrent burst and show one payment.

4. **Failure:** run `make chaos`, kill the DB primary under k6 load, watch failover, and show the invariant checker report zero drift.
5. **Trace:** show one trace spanning gateway, API, bus and consumer.
6. **Close:** open `EVIDENCE.md` and walk one resume claim to its committed run.

Keep fallbacks: a recorded 3-5 minute video and screenshots in the repo, and the lite profile if the full one will not start. Rehearse until it is boring.

8.9 Honest limits (say them before they are asked)

- It simulates payments: a mock PSP, no real money, no card data.
- Numbers are laptop numbers, labelled with their hardware.
- Single-node Redpanda and one Postgres primary with a replica; this is a local reference system, not a production deployment.
- In a real deployment you would change: managed Postgres, three brokers, a real PSP sandbox, a secrets manager, and multiple availability zones.
- No live URL: the README leads with the architecture diagram, the demo video and a results table, and a free GitHub Pages site built from the evidence folder is optional.

9. Interview defence: questions this project lets you answer

- *How do you prevent double charging?* Idempotency keys with request hashing, compare-and-set state transitions, PSP reference idempotency, and the concurrent tests that prove it.
- *What if the process crashes after the DB commit but before publishing the event?* Outbox: the event is in the same transaction, and the relay publishes after restart. Show the crash test.
- *Why not call the PSP inside the transaction?* Connection starvation, long locks, and no way to roll back an external side effect.
- *PSP times out. Did the customer get charged?* Unknown outcome; the state stays `AUTHORIZING` until the resolver queries the PSP by reference.
- *Why double-entry?* Self-checking books; any bug shows up as an imbalance instead of silent corruption.
- *Why add Kafka when Postgres can queue?* Fan-out to independent consumers, replay, per-payment ordering and isolation of slow consumers. The cost is another system to operate, which is why the outbox keeps Postgres authoritative.
- *Polling outbox or Debezium?* Quote your measurements. Polling is simple but adds DB load and latency; CDC lowers both but adds Connect and replication-slot operations, and a stalled consumer can bloat WAL.

- *How do you keep ordering?* Key by `payment_id`, carry a per-payment `event_seq`, and make consumers order by it, because retries can reorder deliveries.
- *Exactly-once?* Not in delivery. At-least-once delivery plus idempotent consumers gives exactly-once effects.
- *What if Redis is down or stale?* Postgres is authoritative; Redis only changes latency, and you measured the difference.
- *What happens when the DB primary dies?* CloudNativePG promotes the replica, in-flight transactions fail, clients retry with the same idempotency keys, and the drill shows zero drift afterwards.
- *Why a modular core instead of microservices?* The money path must commit atomically, so splitting it creates distributed transactions. Only separable consumers are extracted.
- *Why TLA+?* It finds protocol bugs before code exists, and the same invariants then drive the tests.
- *How would you scale this 100x?* Shard by merchant, partition ledger entries by time, add brokers with replication factor 3 and more partitions, shard hot fee accounts, keep reads on replicas and the read model, and keep strong consistency within a shard.
- *Why isn't it deployed to the cloud?* It is a local reference system whose claims anyone can reproduce with one command. In production you would change managed Postgres, three brokers, a real PSP sandbox, a secrets manager and multiple availability zones.
- *How can I trust your numbers?* Every number links to a committed run folder with the hardware, versions, seeds and raw output, CI reruns the suites, and the A/B comparisons hold on any hardware.

10. Scope guard

The project is impressive because it is **deep, tested and honest**, not because it is large. Build it as vertical slices and keep `main` runnable at every step: first the single-service money path with its invariant checker and concurrency tests, then the outbox and webhooks, then the bus and extracted consumers, then Redis and the read model, then the k3d full profile with failover drills, and the evidence pipeline throughout. Each component ships with its measurement; one with no measured result gets cut or documented as a trade-off instead of staying in to pad the stack. Do not cut the tests.

11. Definition of done

The core is what makes the project credible; the stretch items are what make it stand out. Do not start a stretch item before the core is green.

Core

- [ ] Idempotent payment API, state machine, double-entry ledger and invariant checker
- [ ] Concurrency, crash and PSP-fault tests passing, including the naive-implementation contrast
- [ ] Outbox, bus and webhook service with retry topics, DLQ and idempotent consumers
- [ ] Read model with a rebuild-from-scratch check
- [ ] `make up-lite` and `make up-full` work from a clean clone
- [ ] `make chaos`, `make load` and `make verify` print PASS/FAIL and write evidence
- [ ] `EVIDENCE.md` maps every resume claim to a committed run
- [ ] CI green, with nightly artifacts
- [ ] README with architecture diagram, demo video, results table and honest limits

Stretch

- [ ] Debezium relay and the head-to-head benchmark against polling
- [ ] Redis fast path with a measured A/B result
- [ ] TLA+ spec model-checked with TLC
- [ ] Go gateway in front, with one trace from gateway to consumer
- [ ] Argo CD and a canary rollback in k3d
- [ ] GitHub Pages site built from the evidence folder